CM GROUP 04 • PYTHON + FLASK MINI PROJECT

Campus Complaint and Maintenance Portal

Report it. Track it. Resolve it.

PRESENTED BY

Kush	2026UCD3136
Kinshuk	2026UCD2173
Akshat	2026UCD3123

Netaji Subhas University of Technology | 2026

COMPLAINT LIFECYCLE

Every complaint moves through three tracked statuses.

Pending
Complaint logged through the form — the default status for every new record

In Progress
Status updated while maintenance work is under way

Resolved
Resolution notes record the action taken

Python · Flask · SQLite · HTML · CSS · JavaScript

Why a complaint portal?

Facility problems are easy to report informally — and just as easy to lose track of.

THE PROBLEM

Campus issues need a simple way to be reported and followed up. When reports are handled informally, issue details are scattered and it is hard to tell whether a problem is pending, being addressed or already resolved.

TYPICAL ISSUES

- Electrical faults
- Plumbing
- Internet
- Damaged furniture
- Cleanliness

OUR OBJECTIVES

01 Provide a structured form to submit campus maintenance complaints.

02 Categorise and display complaints, with filtering by category and status.

03 Update status and resolution notes, with dashboard totals showing overall progress.

One central place to register, categorise and follow up campus maintenance complaints.

What the portal does

Six features, all served from a single page backed by a Flask JSON API.

Lodge a complaint

Name / roll no., issue summary, category, location and a detailed description — all required.

Six categories

Electrical, Plumbing, IT & Internet, Furniture / Carpentry, Cleanliness and Other.

View & filter

Complaint cards, newest first, filtered by status and by category on the server.

Update status & notes

A dialog moves a complaint between Pending, In Progress and Resolved and records the action taken.

Delete records

Remove a complaint after a confirmation prompt in the browser.

Dashboard counts

Total, Pending, In Progress and Resolved counts, fetched from the backend after every change.

Technologies and their purpose

A lightweight client–server web application with persistent storage.

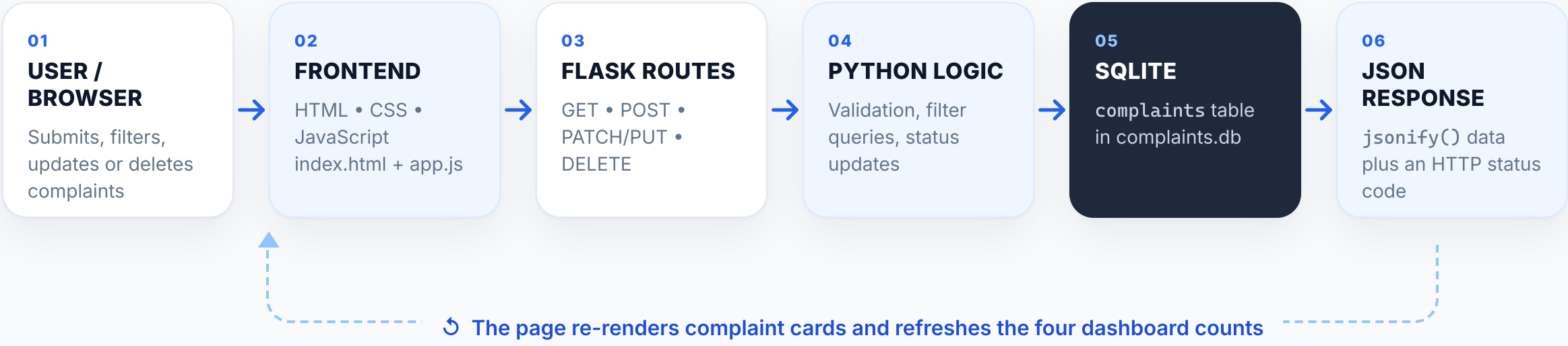
Python	Backend logic, validation and database operations
Flask 3.0.0	Web framework — serves the page and handles API routes
SQLite	Persistent complaint storage via Python's built-in `sqlite3`
HTML	Structure of the dashboard, form, list and update dialog
CSS	Cards, status colours and a responsive two-column layout
JavaScript	Form handling, filters and `fetch()` API requests
JSON	Request and response format between browser and Flask

PROJECT STRUCTURE

```
project/
├─ app.py # routes
├─ database.py # SQLite
├─ complaints.db
├─ requirements.txt
├─ templates/
│  └─ index.html
└─ static/
   ├─ css/style.css
   └─ js/app.js
```

How the frontend and backend communicate

JavaScript `fetch()` calls Flask routes; Flask reads or writes SQLite and replies in JSON.



ON PAGE LOAD `fetchStats()` and `fetchComplaints()` run when the DOM is ready

STATUS FLOW Pending → In Progress → Resolved

One table stores every complaint

SQLite table `complaints`, created by `init_db()` in database.py when the app starts.

FIELD	TYPE / CONSTRAINT	PURPOSE
`id`	`INTEGER PRIMARY KEY AUTOINCREMENT`	Unique complaint ID
`title`	`TEXT NOT NULL`	Short issue summary
`category`	`TEXT NOT NULL`	Complaint category
`location`	`TEXT NOT NULL`	Campus location / room
`description`	`TEXT NOT NULL`	Detailed description
`reporter_name`	`TEXT NOT NULL`	Name / roll no. of reporter
`status`	`TEXT DEFAULT 'Pending'`	Current workflow status
`resolution_notes`	`TEXT DEFAULT ''`	Action taken
`created_at`	`TIMESTAMP DEFAULT CURRENT_TIMESTAMP`	Creation time

HOW IT IS USED

- The table is created only if it does not already exist, so restarting keeps data.
- New rows get status **Pending**, an empty note and a timestamp by default.
- `sqlite3.Row` lets each row become a dictionary for `jsonify()`.
- Every value is passed through `?` placeholders (parameterised SQL).

Seven routes cover the full CRUD cycle

One page route plus six JSON API routes using GET, POST, PATCH/PUT and DELETE.

ENDPOINT	METHOD	PURPOSE
`/`	GET	Render the single-page interface (index.html)
`/api/stats`	GET	Total, Pending, In Progress and Resolved counts
`/api/complaints`	GET	List complaints, newest first; optional `status` and `category` filters
`/api/complaints`	POST	Validate and create a new complaint
`/api/complaints/<id>`	GET	Return one complaint, or 404 if it does not exist
`/api/complaints/<id>`	PATCH PUT	Update status and/or resolution notes
`/api/complaints/<id>`	DELETE	Delete a complaint record

200
Read, update or delete succeeded

201
New complaint created

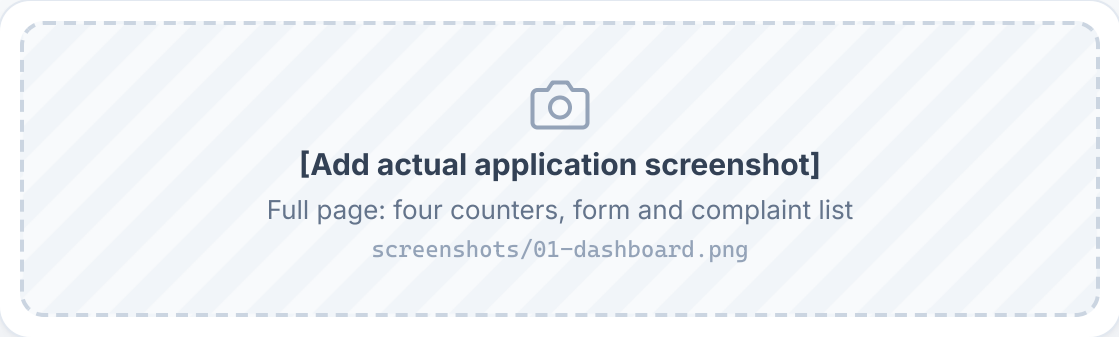
400
Request body empty or missing

404
Complaint ID not found

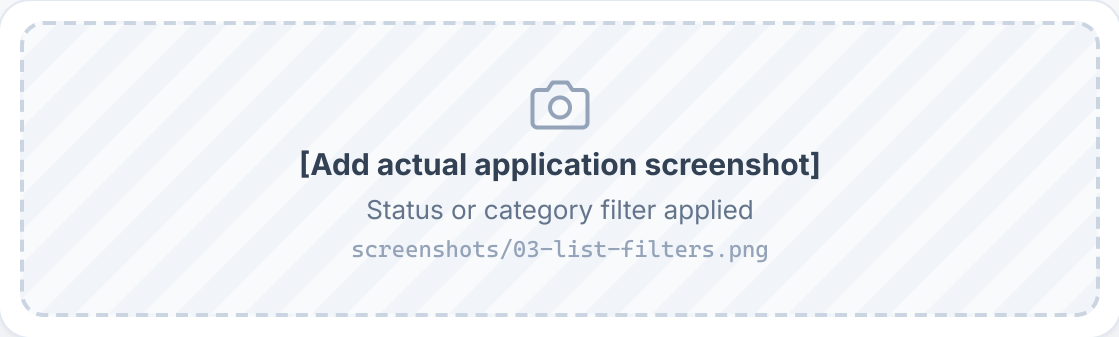
422
A required field is blank

Application walkthrough

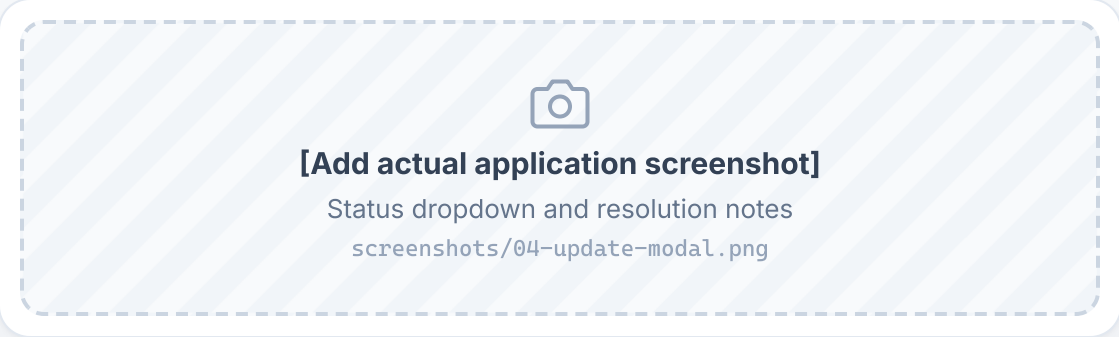
Screens from the running local Flask app at 127.0.0.1:5000.



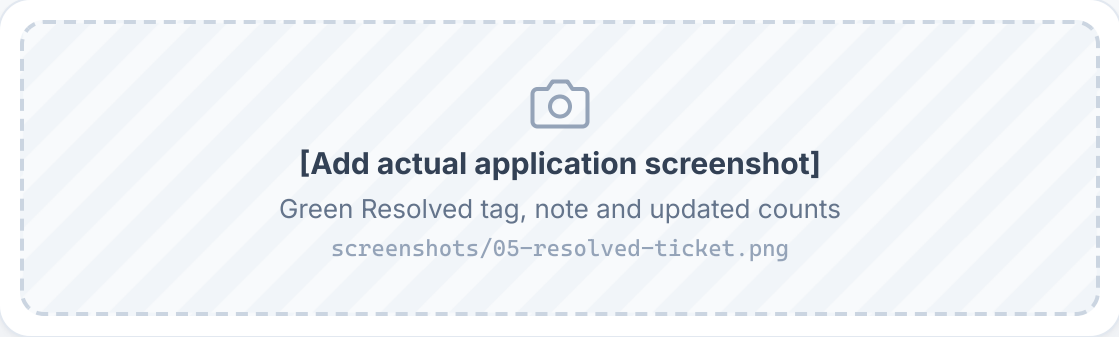
Dashboard and complaint form



Complaint list with filters



Update-status dialog



Resolved complaint with note

Testing the main features

Test cases with expected results, covering every route and user action.

ACTUAL RESULTS: TO BE RECORDED AFTER TESTING

CREATE & VALIDATE

Submit with all five fields
Expect 201, new Pending card, Total +1

Leave a field blank
Browser blocks the form; API returns 422

POST an empty JSON body
Expect 400 with a JSON error

Home page loads
Counters and list appear

READ & FILTER

Filter by each status
Only matching complaints shown

Filter by each category
Only matching complaints shown

GET one complaint by ID
Expect 200 with that record

Request a non-existent ID
Expect 404 “Complaint not found”

UPDATE & DELETE

Pending → In Progress → Resolved
Tag colour and counts change

Save resolution notes
Note appears on the card

Delete after confirming
Card disappears, Total −1

Compare counts with the list
Dashboard totals always match

Where the portal goes next

The current version is a focused complaint tracker for a local demonstration.

CURRENT LIMITATIONS

- No login or roles — anyone using the page can update or delete
- Complaints are not assigned to staff or departments
- No notifications, photo uploads or status history
- Server checks that fields are present, not their allowed values
- Runs on Flask's development server for local use

FUTURE SCOPE

- Student and maintenance-staff logins with permissions
- Assign complaints to a department or staff member
- Email or in-app alerts when a status changes
- Photo attachments, priority levels and keyword/date search
- Status history, downloadable reports and campus-server deployment

Report it. Track it. Resolve it.

A Flask + SQLite portal that turns scattered maintenance requests into trackable records.

WHAT WE BUILT

A complete create–read–update–delete web application: a complaint form, filterable complaint cards, a status-update dialog and live dashboard counts, backed by six JSON API routes and a SQLite database.

WHAT WE PRACTISED

Flask routing and HTTP methods, SQLite with parameterised queries, JSON APIs, `fetch()` with async/await, DOM updates and responsive CSS layouts.

THANK YOU

Questions?